

《面向对象程序设计》之——

习 题

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

- **友元函数求矩形去掉所包含的最大圆后的面积**
- 设计一个类Rectangle表示矩形，其中包括私有成员变量width和height分别表示矩形的宽度和高度。请实现一个友元函数CalculateArea，计算矩形面积减去其所包含的最大圆后的面积。pi取值为3.14。在友元函数中完成输出，结果保留两位小数(五舍六入)（格式输出控制如下：`cout<<fixed<<setprecision(2)`）。并在主函数中进行测试。

```
#include <iostream>
#include <iomanip>
using namespace std;

class Rectangle {
private:
    double width;
    double height;
//类的定义未完待续

/* 请在这里填写答案 */

int main() {
    double width, height;
    cin >> width;
    cin >> height;

    Rectangle rect(width, height);
    CalculateArea(rect);

    return 0;
}
```

输入样例:

在这里给出一组输入。例如:

```
4 6
```

输出样例:

在这里给出相应的输出。例如:

```
11.44
```

```
#include <iostream>
#include <iomanip>
using namespace std;

class Rectangle {
private:
    double width;
    double height;
//类的定义未完待续

/* 请在这里填写答案 */

public:
    Rectangle(double w, double h) : width(w), height(h) {}

    friend double CalculateArea(Rectangle& rect);
};
```

```
double CalculateArea(Rectangle& rect)
{
    double r,area;

    r=(rect.width<rect.height)?rect.width:rect.height;
    r=r/2.0;

    area=(rect.width*rect.height)-(r*r*3.14);

    cout<<fixed<<setprecision(2)<<area<<endl;

    return 0;
}
```

题目描述

通常数组一旦申请，其大小就不可变化。但实际应用中，数组元素个数变化较大，因此，有必要设计一种自适应数组，可以随着存储数组元素的实际需要动态扩容或者缩小容量，以增加空间利用效率。

为此，请设计一个自适应数组模板类`scale_array`，支持以下功能：

- (1) 初始化时根据指定长度申请动态数组
- (2) 支持`insert(x)`运算：将元素`x`存储至数组的第一个空闲单元中，当数组已满，需先将数组扩容至当前数组长度的 $3/2$ 倍，再进行元素插入
- (3) 支持`lastdelete()`运算：若数组为空数组，则返回“empty”，否则将数组中最后一个非空数组单元存储元素`x`删除并返回，且若元素`x`删除后，有一半及以上数组单元空闲时，将数组长度缩减至当前数组长度的 $3/4$ 倍
- (4) 支持`capacity()`运算：返回当前数组剩余空闲数组单元数量
- (5) 该自适应数组可用于各种标准数据类型，和标准数组一样支持下标运算
- (6) 任何时候数组长度都不小于2

请根据以上要求，完成该自适应数组模板类`scale_array`。

所有输入均合法。

输入格式:

第一行输入一个整数M，代表着接下来有M组操作

每一组操作由若干行操作组成

每一行为一个操作，每行的第一个字符为操作类型，共有5种不同操作。

N为新建一个自适应数组，接着输入两个整数，第一个整数代表数组长度，第二个整数代表数据类型，1为int，2为double，3为char

I为插入操作，接着输入一个数据，为待插入元素，保证输入数据类型合法

D为删除操作

C为查询容量操作

E为结束操作

每组操作第一行肯定是N操作，最后以E操作结束

输出格式:

对于D操作，输出被删除的元素或者empty（数组为空时）

对于C操作，输出剩余空闲数组单元数量


```

3
N 5 1
I 12
I 23
I 34
I 45
I 56
I 67
C
D
E
N 3 2
I 12.34
D
C
E
N 6 3
I A
I B
I C
I D
I E
D
C
E

```

```

1
67
12.34
2
E
2

```

输入格式:

第一行输入一个整数M，代表着接下来有M组操作

每一组操作由若干行操作组成

每一行为一个操作，每行的第一个字符为操作类型，共有5种不同操作。

N为新建一个自适应数组，接着输入两个整数，第一个整数代表数组长度，第二个整数代表数据类型，1为int，2为double，3为char

I为插入操作，接着输入一个数据，为待插入元素，保证输入数据类型合法

D为删除操作

C为查询容量操作

E为结束操作

每组操作第一行肯定是N操作，最后以E操作结束

输出格式:

对于D操作，输出被删除的元素或者empty（数组为空时）

对于C操作，输出剩余空闲数组单元数量

```
#include <iostream>
#include <string>
using namespace std;

template< class T>
class x_array
{public:
    x_array(const int len);
    T& operator[] ( int );
    const T & operator[] ( int )const;
    int getlen()const{ return length; }
    int capacity()const {return maxlength-length;}
    void insert(const T);
    void lastdelete();
    ~x_array();
private:
    int length;
    int maxlength;
    T * num;
};
```

```
template< class T>
x_array<T>::x_array(const int len)
{
    num = new T[len];
    maxlength = len;
    length=0;
}
```

```
template< class T >
x_array<T>::~~x_array()
{
    delete[] num;
}
```

```
template< class T >
T & x_array< T> ::operator[] (int i)
{
    if( i < 0 || i >= length)
        throw string( "out of bounds" );
    return num[i];
}
```

```
template< class T >
const T & x_array< T > ::operator[] ( int i )
    const
{
    if( i < 0 || i >= length)
        throw string( "out of bounds" );
    return num[i];    }

```

```
template<class T>
void x_array<T>::insert(const T x)
{
    if (length<maxlength)
        num[length++]=x;
    else
    {
        maxlength=maxlength*3/2;
        T *temp=new T[maxlength];
        for (int i=0; i<length; i++)
            temp[i]=num[i];
        delete[] num;
        num=temp;
        num[length++]=x;
    }
    return; }

```

```
template<class T>
void x_array<T>::lastdelete( )
{
    if (length>0)
    {
        cout<<num[--length]<<endl;
        if (length<=maxlength-length)
        {
            if (maxlength>2)
            {
                maxlength=maxlength*3/4;
                T *temp=new T[maxlength];
                for (int i=0; i<length; i++)
                    temp[i]=num[i];
                delete [] num;
                num=temp;
            }
        }
    }
    else
        cout<<"empty"<<endl;

    return;
}
```

```
template <class T>
void scalearrayprocessing(x_array<T> & A)
{
    char flag;
    cin>>flag;
    while (flag!='E')
    {
        switch (flag)
        {
            case 'I':
            {
                T x;
                cin>>x;
                A.insert(x);
                break;
            }
            case 'D':
            {
                A.lastdelete();
                break;
            }
            case 'C':
            {
                cout<<A.capacity()<<endl;
                break;
            }
        }

        cin>>flag;
    }
    return;}
}
```

```
int main( ){
    int count=0;
    cin>>count;
    for (int i=0; i<count; i++)
    {
        char flag;
        int length, typeflag;
        cin>>flag>>length>>typeflag;

        if (typeflag==1)
        {
            x_array<int> A(length);
            scalearrayprocessing(A);
        }
        if (typeflag==2)
        {
            x_array<double> A(length);
            scalearrayprocessing(A);
        }
        if (typeflag==3)
        {
            x_array<char> A(length);
            scalearrayprocessing(A);
        }
    }

    return 0;}
```

关于对象数组、对象指针数组（以“车辆保费计算”为例）

```
int main()
{
    int num;
    Vehicle * *p;

    cin>>num;
    if (num>0)
    {
        p=new Vehicle * [num];
        for (int i=0;i<num;i++)
        {
            string v_ID;
            cin>>v_ID;
            char type;
            cin>>type;
            .....

            switch (type){
                case 'T':p[i]=new truck(v_ID,v_p,v_a); break;
                ..... }

            .....
        }
    }
}
```

```
Vehicle** p = new Vehicle*[num];
Vehicle* p =new Vehicle[num];
```



```
int main() {  
    // 学生管理  
    int m;  
    cin >> m;    // 输入学生数量  
    MyStudent* stuList = new MyStudent[m]; // 学生对象数组-要有默认构造  
  
    for(int i = 0; i < m; ++i) {  
        int major, level;  
        cin >> major >> level;    // 输入第i个学生的参数 (专业, 年级)  
        stuList[i] = MyStudent(major, level); // 使用拷贝赋值  
    }.....  
}
```

。 。 。

```
int main() {  
    int n, type;  
    cin >> n;    // 输入学生数量  
  
    MyStu** stus = new MyStu*[n]; // 学生对象指针数组, 不强求有默认构造  
  
    for(int i=0; i<n; ++i) {  
        int major, level;  
        cin >> major >> level;    // 输入第i个学生的参数 (专业, 年级)  
        stus[i] = new MyStu(major, level);  
    }  
}
```

。 。 。

```
Vehicle** p = new Vehicle*[num];  
Vehicle* p =new Vehicle[num];
```

- 客观题:
- **OOP/C++**概念基础、引用、异常处理、命名空间、输入输出流、类和对象、继承、静态成员、多态、重载、友元、链表、模板等
- 非客观题:
类和对象、继承、静态成员、多态、重载、友元、链表、模板等

Thank you !